

110.2. La fonction lirePanneau() — lire une case du panneau

#include <lirePanneau> : `lirePanneau(colonne, ligne)` rend la tuile posée dans cette case du panneau.

1. Ce qu'il faut comprendre

Ce qu'elle fait : comme `lire()`, mais **dans le panneau** : la tuile d'une de ses cases, 0 si elle est vide.

La ligne à écrire : `#include <lirePanneau>`.

Ses arguments : la colonne et la ligne, depuis le coin du panneau.

Ce qu'elle coûte : environ **90 octets** la première fois.

Essaie : lis la case (5, 0), vide : `t` vaut 0.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **423 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      52 o ← la console, à l'allumage – la mise en route de la console
main()         44 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireNombre – attend de pouvoir écrire une case
EcrireNombre   33 o ← nombre() – écrit un nombre en chiffres
CopierTuiles   40 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         176 o ← CopierTuiles – 11 lettres sur 43
```

Lettres gravées : C0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
panneau()      x 1   15 o sur place, dans main()
nombre()       x 1   10 o sur place, dans main() + EcrireNombre (33 o)
poserPanneau() x 1    6 o sur place, dans main()
lirePanneau()  x 1    4 o sur place, dans main()
image()        x 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <lirePanneau> : lire une case du panneau ----

#include <poserPanneau>
#include <ALPHABET>
#include <panneau>
#include <lirePanneau> // lirePanneau() : la tuile d'une case du panneau
#include <nombre>
```

2. UNE VARIABLE — T

```
uint8_t t = 0;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poserPanneau(2, 0, ALPHABET[2]); // un C : la tuile 3
    panneau(0, 128);
    t = lirePanneau(2, 0);           // t reçoit 3
    nombre(2, 4, t);                // et on l'écrit sur le fond : 003

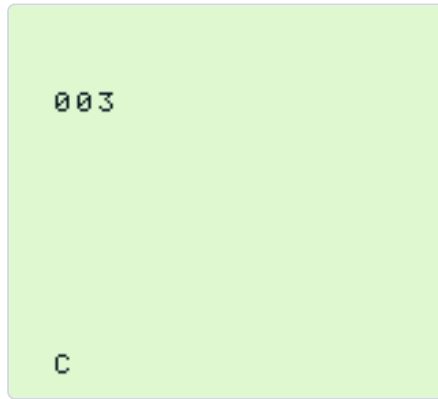
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un C dans le panneau, et 003 écrit sur le fond : son numéro.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ t vaut 3, le numéro du C (3)

✓ le fond montre 003

Ensuite : **110.3. Qui passe devant ? Les trois couches de l'écran** — chapitre 17 : Le mouvement : les lutins au pixel près. ·

gameboy3 — écrire une cartouche Game Boy en C++